SMARTATTEND

Technology Stack

Tools and components for the SmartAttend solution

Project documentation • Requirement Analysis

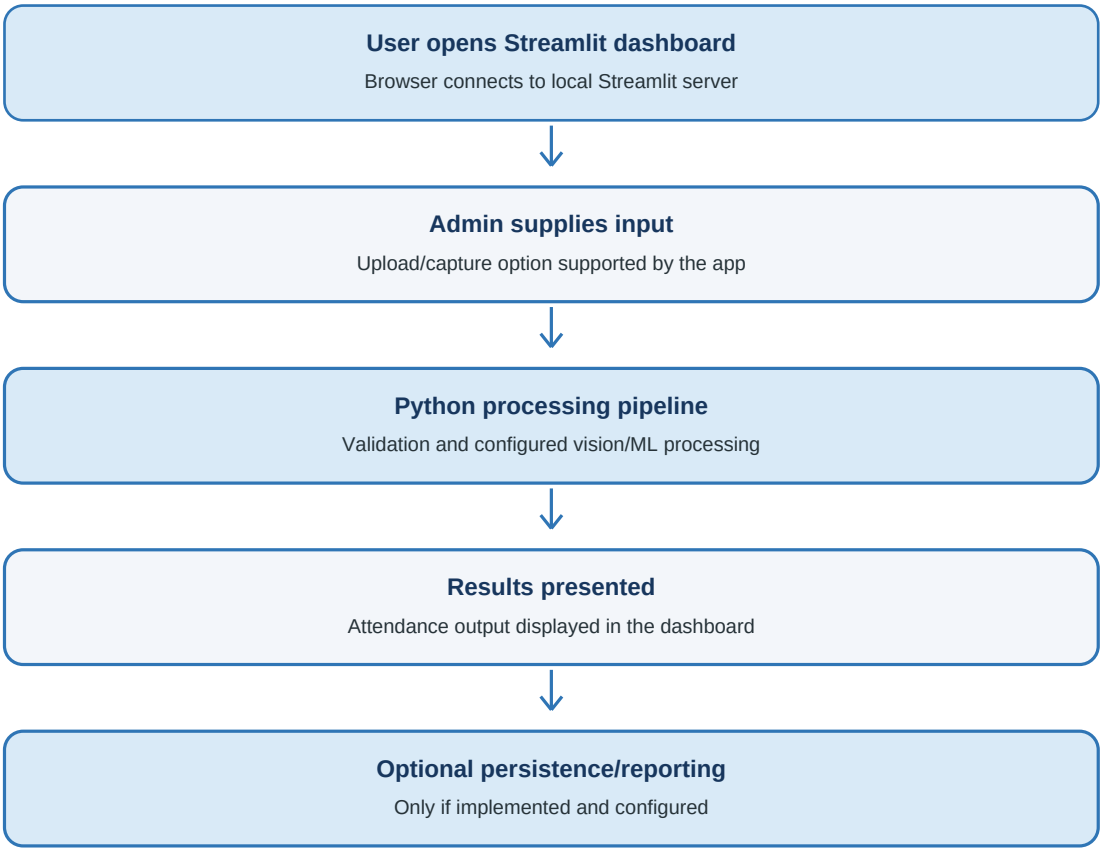
Prepared for an AI-assisted attendance management application

1. Technology overview

The following table distinguishes the technologies confirmed by the current setup from components that must be verified in the repository before final submission.

Layer	Technology / component	Purpose	Status / note
User interface	Streamlit	Browser-based dashboard and user interactions	Confirmed by the running app at localhost:8501
Programming language	Python	Application logic and processing pipeline	Confirmed by project setup
Environment	Python virtual environment (.venv)	Isolates project dependencies	Used in local setup
Package management	pip + requirements file (if present)	Installs Python dependencies	Use the requirements file from the actual project root
Computer vision	OpenCV or project-specific vision library	Potential image/video frame processing	Verify imports and dependencies in repository
Face recognition / ML model	Project-specific model or library	Potential face detection/identity matching	Verify model, enrollment flow and license
Data storage	Local files / database / Supabase	Potential attendance persistence	Do not claim a specific store until code/config confirms it
Reporting	CSV / dashboard summaries	Potential attendance export and summaries	Verify implemented export features
Development tools	Visual Studio Code, PowerShell	Editing and running the application	Used in local setup
Version control	Git and GitHub	Source-code management	Repository source should be documented

2. Runtime flow



3. Environment and run commands

Create and activate an environment from the actual project root:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Install dependencies if a requirements file exists:

```
python -m pip install -r requirements.txt
```

Run the Streamlit entry file specified by the README (example only):

```
python -m streamlit run app.py
```

4. Verification checklist

Before final submission, record the actual Python version, dependency versions, model/library names, storage mechanism, supported input types, and exact Streamlit entry file. Update this document to match the implementation rather than listing unverified technologies as completed.